# CS542200 Parallel Programming
# Homework 1: Odd-Even Sort

張原鳴, **113062571**

## REPORT

### Implementation

- How do you handle an arbitrary number of input items and processes?

A:

我用 **block decomposition with remainder** 的方式處理任意長度的輸入陣列與任意數量 MPI process，分配每個process 他該負責的連續元素 (chunk):

給定總元素數 N 與總 process 數 P

- 平均區塊大小 q = N / P，餘數 r = N % P
  - 若 rank < r，就分配 q + 1 個元素；
  - 否則分配 q 個元素。
- 設定每個 process 所負責的 chunk 從 index **start** 開始。

```
int rank_size = N/size + (rank < N % size ? 1 : 0);
int start = rank * (N / size) + (rank < N % size ? rank : N % size);
```

- How do you sort in your program?

  在 sort 的規劃上，我分成兩個階段: local sort 和 global sort。

  在分配好每個 process 要負責

  - Local sort: 在最一開始，將每個 rank 負責的 chunk 做 local sort，我使用 **Boost Spreadsort** 的 float_sort() 對每個 process 的 chunk 獨立進行排序。
    - Boost Spreadsort 結合 radix sort，平均速度可以接近 O(N)。
  - Global sort: 多個 phases 讓各 process 的區塊互相 communicate。
    - **Even Phase**：偶數 rank 與右邊的鄰居交換並部分合併；奇數 rank 與左邊鄰居交換並部分合併。
    - **Odd Phase**：奇數 rank 與右邊的鄰居交換；偶數 rank 與左邊的鄰居交換。

在每個 phase 中，當兩個 ranks 交換各自的資料時：

- 在傳輸不同 processes 之間的資料時，我使用 **MPI_Sendrecv**，這樣可以同時進行送出與接收，避免兩邊都在等對方先傳資料而造成 **deadlock**，並且保證鄰居之間的資料交換是安全且同步的。
- 我在程式中使用自己設計的 **partial merge** 方法：左邊的 rank 只保留 最小的 **n** 個元素，右邊的 rank 則保留 最大的 **n** 個元素，藉此只交換與保留必要的資料，避免做出完整的 merge，減少了運算與記憶體的開銷，同時也能維持正確的全域排序。
- 這樣每次做完 merge，兩邊 rank 各自的 chunk 一定都是 sorted。

**Other Efforts**

在 I/O 使用 MPI_COMM_SELF

- 檔案輸入與輸出(MPI_File_read_at, MPI_File_write_at) 採用 MPI_COMM_SELF，而非 MPI_COMM_WORLD。
- 使用 MPI_COMM_SELF 時，每個 rank 都擁有獨立的檔案 handle。由於每個 rank 透過座標運算已經能確定自己負責的資料區間，因此只需依照自己的 offset 進行讀寫，無需 collective 同步。
- 可減緩 I/O overhead。

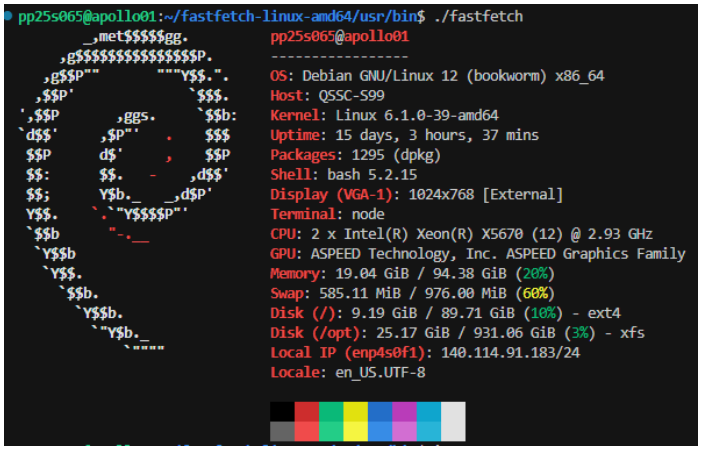
**Experiment & Analysis:**

Explain how and why you do these experiments? Explain how you collect those measurements? Show the results of your experiments in plots, and explain your observations.

**In the experiment, I implemented testcase 33 in all cases.**

## i、 Methodology

### (a). System Spec



**(b) Performance Metrics**: How do you measure the computing time, communication time and IO time? How do you compute the values in the plots?

- Execution (CPU) Time
  - Local sort
  - Partial merge
  - MPI_Init, MPI_Finalize
  - Others
- Communication Time
  - MPI_Sendrecv
  - MPI_Barrier
- I/O Time
  - Input: MPI_File_open, MPI_read_at, MPI_File_close.
  - Output: MPI_File_open, MPI_write_at, MPI_File_close.

For each case, I measure the mean performance on ranks, running three times and getting the average of times.

## Plots: Speedup Factor & Profile

### Experimental Method:

- Test Case Description: Explain the test data and its sizes you've chosen.
- Parallel Configurations: Describe the number of processes used, and how nodes and cores are distributed.

A: 我測試的資料集是第 33 個 case, N (element number) = 536869888

在這個實驗, 我嘗試了兩個版本（單節點和多節點）

- Single node: 1/2/4/8 processors
- Multi nodes: 我保持了每個節點分配到 4 個 processors (除了 N1n1).
  - 1 node 1 processor (N1n1)
  - 1 node 4 processors (N1n4)
  - 2 nodes 8 processors (N2n8)
  - 4 nodes 16 processors (N4n16)

將 N1n1 的執行時間除上後三個 cases 的執行時間就能分別得到各個 case 的 speedup factor.

$$S(p) = \frac{T_s}{T_p}$$

## Performance Measurement:

- Use a profiler (such as perf, nsys, mpiP) for performance analysis.
- Provide basic metrics like execution time, communication time, IO time, etc.

A: I Use a **nsys profiler** to measure the statistics

**Basic Metrics**

- **Execution time**：從程式開始到結束的總時間。
- **Computation time**：程式中與排序、合併等本地計算相關的時間。
- **Communication time**：MPI 通訊相關操作所花費的時間。
- **I/O time**：包含輸入與輸出檔案的時間。

## Analysis Results:

- Display results generated by the profiler; this could be in the form of tables, charts, or other visualization tools.
- Identify Performance Bottlenecks: Use the profiler's data to point out which parts of the code or which communication patterns are creating performance issues.
- Conduct strong scalability experiments, and plot the speedup and time profile results as shown in figure 1 and figure 2.

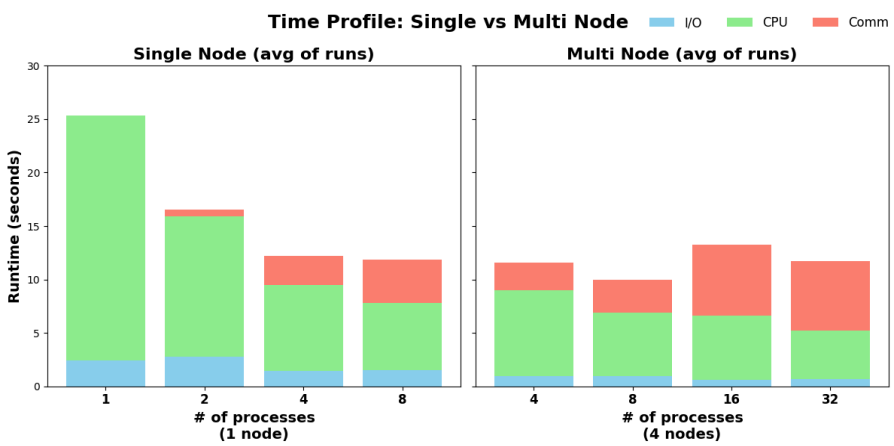



**Figure 1. Time Profile**

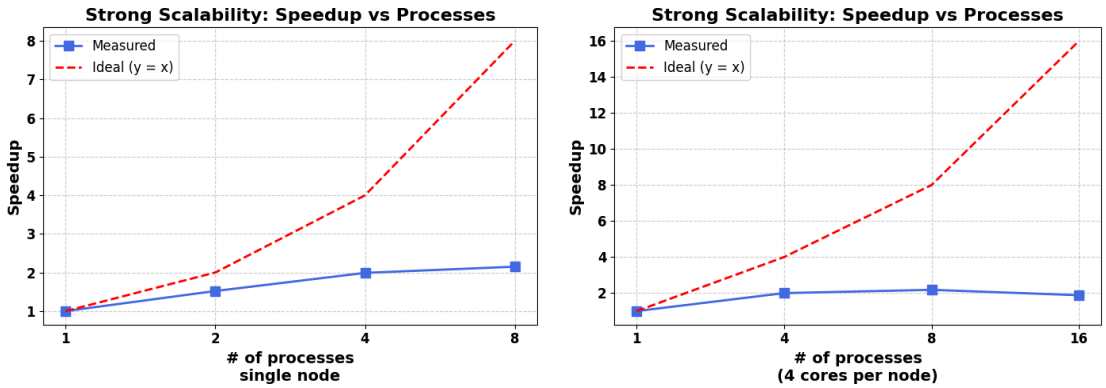



**Figure 2. Speedup Factor**

**Discussions:**

1. Figure 1 (Time Profile)
    a. Single node
        i. 當 process 數從 1 增加到 8，總執行時間顯著下降，主要是因為 **CPU** 計算時間隨著平行化明顯減少，算是達到平行化的目的。
        ii. 然而，通訊時間（communication time）隨著 process 數上升,兒逐漸增加，形成了主要的 bottleneck。
    b. Multi nodes
        i. 隨著process數上升，**總執行時間並沒有持續下降，反而在** process **數較多** (16, 32) **時出現** communication time **大幅上升。**
        ii. **依照推測，在需要跨節點的情形下，**communication overhead **是比較顯著的。**
        iii. **此外，**computation time **在** process **數變多時的效能增進並不明顯，也是造成整體效能較差的原因之一。**
    c. I/O **時間在不同設定下差異不大，顯示** I/O **並非主要瓶頸。**
    d. 針對這個 strong scaling 的實驗可以看到針對單節點，4 個 processes 大概就是這個程式的 sweet pot。再往上，communication overhead 會漸漸抵消平行化的效益。
2. Figure 2 (Speedup Factor)
    a. Single node
        i. 在 8 processes 時僅獲得不到 3 倍加速，效率約 35% 左右。
    b. Multi nodes
        i. 在 16 processes 以後，speedup 不再上升，甚至略有下降。
        ii. 搭配 Figure 1 的結果，清楚顯示跨節點的 **communication overhead** 已成為 bottleneck，導致增加節點數並不能帶來實際效能提升。
    c. 理想 speedup 為線性 (y = x)，但實測結果明顯低於理想值。

**Optimization Strategies:**

在後續的效能優化應主要分為兩個方向

- 加強平行化的效果，讓 computation time 隨著 processor 上升時能顯著下降。
- 原本在兩個相鄰的 rank 做 data merge 時雖然是同時傳送資料 (MPI_SendRecv)，但資料是全送給對方的，若能針對這點改進，在 SendRecv時嘗試不傳送大量的資料就能達到排序效果，應該能大幅改進 communication time 所造成的 bottleneck。

1. 邊界檢查 + Binary Search 找切點
   a. 原本每次 partner 間必定交換完整的資料，再透過 two pointer merge 讓兩邊各自拿前半和後半部。資料傳輸量大（兩邊都要傳輸所有的資料）。
   b. 方法分成兩部份：
      i. 每個 phase 讓左邊的 rank 拿出自己最右邊（left_max）的值，右邊的 rank 拿最左邊的值 (right_min)。兩邊透過 **MPI_SendRecv** 讓這兩個值做比較，若 left_max <= right_min，則代表這兩個 chunk 已經排序好了，不用做到 merge。
      ii. 否則，將 left_max 在右邊的 chunk，right_max 在左邊的 chunk 各自做 binary search 找出一個在 chunk 中的切點（cut_idx)。在左邊的 chunk 中，cut_idx 右邊的資料傳到右邊的 chunk，而在右邊的 chunk 中，cut_idx 左邊的資料傳到左邊的 chunk，使用 **MPI_SendRecv**。
   c. 想法：以左邊 chunk 為例：若今天 一個 phase 中 merge 完之後，左邊 chunk 原本的最大值（prev_max）一定 >= merge 完的最大值 (cur_max)，代表比 prev_max 大的值一定最後會在右邊的 chunk 那就沒有傳到左邊 chunk 的必要。反之右邊 chunk 也成立。

2. 記憶體結構優化
   a. 每個 phase 的 merge 過程會建立新的 vector 作為暫存，並透過 push_back 持續插入元素，這個作法會造成一些效能上的問題：
      i. 每次宣告一個新的 buffer 時，都要重新配置一個記憶體 (malloc) 在heap，並且 phase 結束時又要釋放這塊空間 (free)。若又沒有預定義好結構的大小且又沒用 reverse() 或 resize()，在 push_back 的時候又可能因為 size 不夠要 reallocate 新的記憶體。
      ii. 主函式中，我們針對排序與通訊過程中需要重複使用的暫存區，事先配置了三個固定大小的 buffer，並在每個 phase 中重複使用。
         1. d：儲存本 rank 的 data chunk。
         2. recvbuf：用來接收來自 partner rank 的交換資料。

3. keep：在 merge 階段用來暫存保留下來的元素。

這些 buffer 在程式一開始就依據輸入資料大小 N 和 process 數 size 計算最大可能需求，並使用 numa_alloc_onnode 在對應的 NUMA 節點上配置。

iii. **numa_alloc_onnode** 會將記憶體配置在與目前 CPU 所屬的 NUMA 節點綁定的 local memory 上，可以避免 remote access。rank 在做排序或 merge 時，CPU 都是存取 **local memory**，減少 computation time。

3. 優化 partial merge
   a. 右 rank 的 merge 過程是從 尾端往前挑最大元素，結果 push 到 keep 時會變成「由大到小」。
   b. 原本的作法是直接對整個 buffer 做 reverse。這麼做就會讓每次的 merge 多花 O(N) 的時間來讀取整個 buffer。造成不必要的 computation overhead。
   c. 作法很簡單，我將右 rank 直接用從尾端填回陣列的方式，這樣做在 merge 結束之後就直接是遞增的順序，不用再做 reverse。

## Improvement and Ablation Study

經過了以上的主要改動，並拿來和原本的版本進行比較。

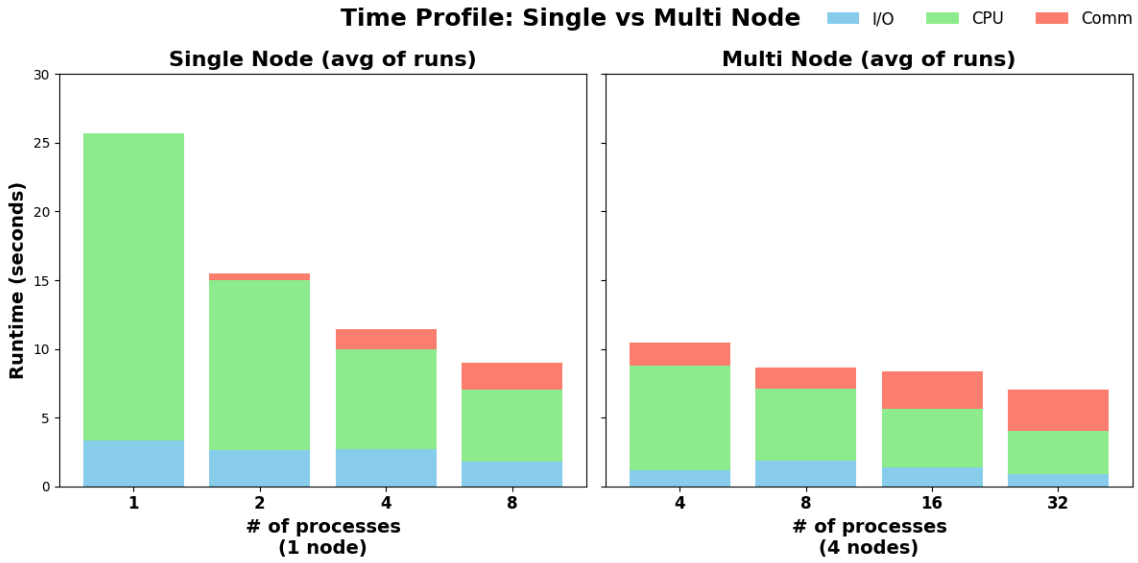



**Figure 3. Comparison on Time Profile Before and After Optimization**

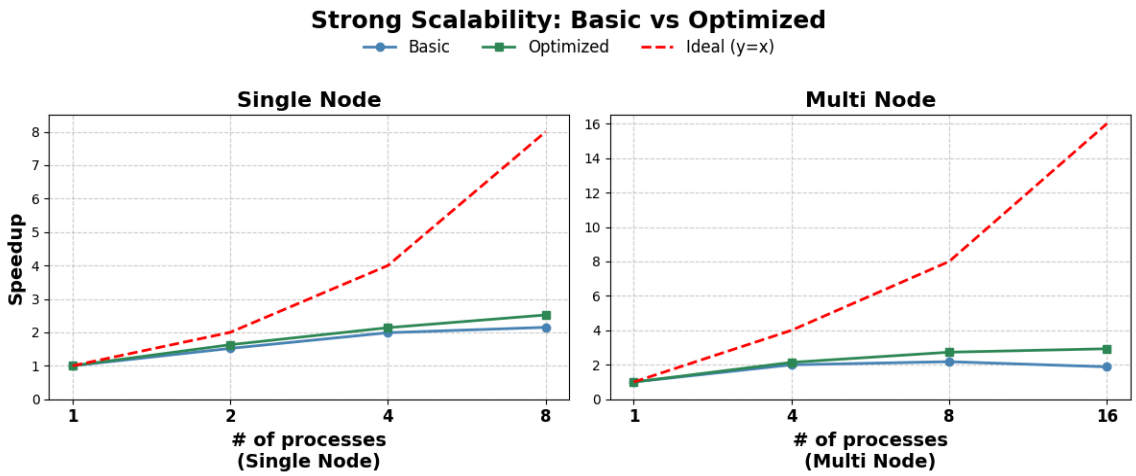



**Figure 4. Comparison on Speedup Factor Before and After Optimization**

## Discussions

### 1. Time Profile

i、 根據改進後的實驗結果，整體的 communication time 是有隨著 processor 數上升而明顯下降，也隨之帶動執行時間的下降，達到了平行化的效果。

ii、 Communication time 在 process 數上升時也沒有像原本上升那麼劇烈。

iii、 此優化應要歸功於邊界檢查和二元搜尋法，減少了資料傳輸量同時減少不必要的 MPI_Sendrecv 資料傳輸又透過縮減資料大小降低 merge 所花的時間。

### 2. Speedup Factor

i、 Optimized 版本的 speedup 明顯優於 Basic 版本，顯示所提出的改進有效降低了 overhead。

ii、 然而，實際 speedup 與理想線性 (y = x) 仍有落差。

### 3. Ablation study

此外，我也做 ablation study 針對邊界檢查和二元搜尋，以 testcase 33 為例

- Case 1: 沒有邊界檢查和二元搜尋
- Case 2: 有邊界檢查但沒有二元搜尋
- Case 3: 有邊界檢查和二元搜尋

**Result (Use time profile):**

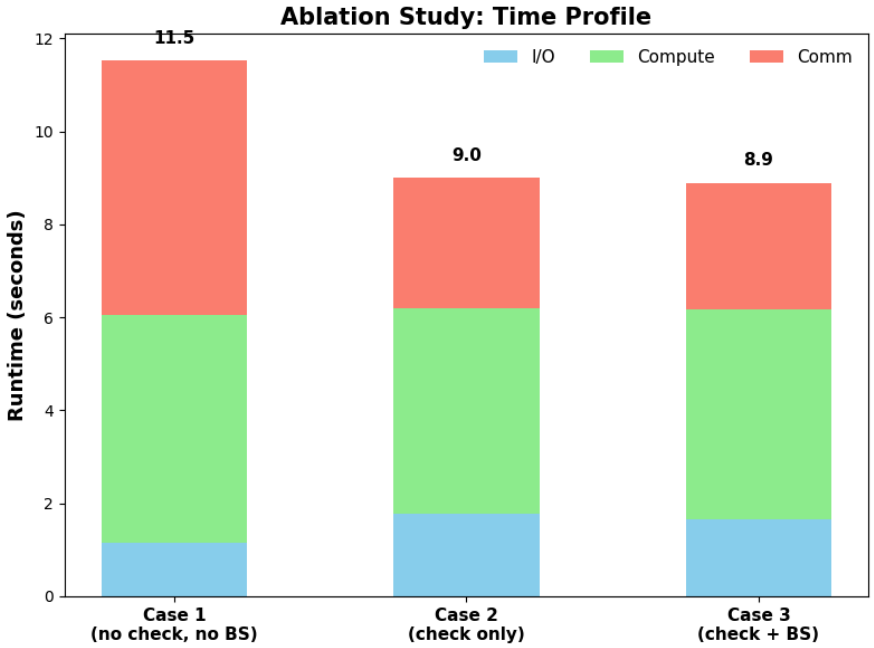



**Figure 5. Ablation Study on Boundary Check and Binary Search**

可以發現，邊界檢查是效能進步的一大關鍵。由此可知在這個 case 上大部分的 phase 其實兩邊的 chunk 都已經排序好了，省下 merge 的時間就可以大幅提升效能。

## Experiences / Conclusion

1. Conclusion of assignment
    a. 針對原 baseline 的效能問題做出優化方案，確實降低了 computation time，也部分改善了 communication。
    b. 經過改善後，針對多節點的 communication time 的比例相對單節點還要高，scalibility 也受到嚴重的限制，可更加確定跨節點的 MPI 通訊會帶來的 overhead 是較高的，也成為了主要的 bottleneck。
2. What have learned in this lab
    a. 學會了如何使用 MPI API 來達成程式的平行化，包括 process 間的通訊 (MPI_Sendrecv、odd-even sort) 與跨節點協作。
    b. 學會了如何使用 MPI I/O（MPI_File_read_at / MPI_File_write_at）進行平行檔案處理。
    c. 學會了分析程式的 time profile，區分 computation / communication / I/O 三類時間，並透過 profiler 找出瓶頸。

3. Difficulties
    a. 如同第一點所說，在多節點環境下，MPI 通訊延遲比想像中更嚴重，尤其在 process 數增加時，scalability 明顯停滯。
    b. 程式 debug 難度高：若出現 wrong answer，常需要花費大量時間追查原因，部分錯誤來源可能是 rank 在讀檔時超出分配範圍或式少配到，導致資料錯位或排序結果錯誤。
4. Future works
    a. 嘗試使用非阻塞通訊 (MPI_Isend/Irecv)
        i. 在目前的實作中，rank 間的資料交換主要使用 MPI_Sendrecv，屬於 **blocking** 通訊。這表示在發送或接收完成之前，程式必須停下來等待，導致計算核心在等待資料時無法繼續工作。
        ii. 若改用 **non-blocking** API，例如 MPI_Isend 與 MPI_Irecv，則可以先發起通訊請求，然後立刻返回繼續執行其他計算，達到執行時間 overlap 的效果。
        iii. 對多節點 (multi node) 可能會有幫助，因為網路 latency 本來就是主要瓶頸。透過重疊，能在不降低計算效率的情況下掩蓋部分通訊延遲，進而改善 scalability。
    b. 使用 weak scaling 實驗
        i. Weak scaling 固定每個 process 的工作量，總問題大小隨 process 數成比例增加，可以提供更完整的 scalability 評估。
        ii. 由於若要增大資料規模需要生成新的資料，目前想到比較可行的方法是以原 case 為最大資料的基準往下取樣資料來做。